

Architectural Blueprint and Engineering Specification: Python Abstract Syntax Tree Analysis for DRAKON Diagram Generation

The evolution of automated visual programming platforms necessitates a paradigmatic shift from rudimentary lexical text matching to rigorous, syntax-aware structural analysis. The ai-dragon project currently relies on regular expressions—such as `/(?:function\s+\w+|const\s+\w+\s*=\s*(?:async\s*)?(?!(def\s+\w+|async\s+def\s+\w+))/g`—to identify execution boundaries within JavaScript and TypeScript repositories. While this approach suffices for basic function identification, it is mathematically and computationally incapable of parsing context-free grammars. Regular expressions cannot reliably extract nested control flow structures, variable scoping, or complex error-handling routines inherent to the Python programming language.

To achieve the objective of generating mathematically sound, topologically precise DRAKON Intermediate Representation (IR) graphs, the architecture must implement a deterministic parsing pipeline based on Abstract Syntax Trees (AST). The visual language DRAKON, originally developed for the aerospace industry, operates on strict cognitive and topological constraints: the absolute prohibition of line intersections, the preservation of the central vertical axis (the "Happy Path" or Skewer), and the deterministic routing of secondary logic to the rightward periphery.¹ This report provides an exhaustive engineering blueprint for integrating Python AST analysis into the ai-dragon Cloudflare Worker environment, detailing the required Control Flow Graph (CFG) extraction methodologies, execution environment evaluations, specific structural mapping rules, complexity estimation metrics, and incremental processing strategies for large codebases.

Python AST for Control Flow Extraction and DRAKON Semantics

The extraction of control flow from Python source code requires traversing a hierarchical representation of the program's structure. The Hybrid Topological Semantic Extraction (HTSE) paradigm dictates that semantic logic extraction must be strictly separated from geometric layout calculation.¹ The extraction phase focuses solely on defining the logical relationships—represented as next, yes, and no pointers—which a downstream deterministic layout engine (such as `drakonwidget.js`) subsequently translates into orthogonal Manhattan routing coordinates.¹

The Mechanism of AST Traversal and Basic Block Generation

Whether leveraging Python's native ast module or a language-agnostic parser like tree-sitter, the fundamental process remains the generation of a tree where each node represents a syntactic construct. Python's native ast module provides classes derived from the `_ast` C module, such as `ast.If`, `ast.For`, and `ast.Try`.² Traversing these structures requires implementing the Visitor pattern, typically by extending `ast.NodeVisitor`, which allows the execution of specific logic upon encountering distinct node types.³

During this traversal, the analyzer must group sequential, non-branching statements—such as multiple `ast.Assign` or `ast.Expr` nodes—into a single "basic block".⁴ In the context of DRAKON, these basic blocks translate into singular action nodes. Tools like `pyflowchart` implement this concept through parameters such as `simplify`, which automatically collapses one-line conditional bodies into a single visual node, effectively executing "orphan trimming" and reducing visual noise.¹ Without this simplification, the resulting DRAKON diagram would suffer from severe visual clutter, counteracting the language's core tenet of cognitive ergonomics.¹

Mapping Conditional Branching (If/Elif/Else)

Conditional logic in Python is encapsulated within the `ast.If` node, which contains three primary attributes: `test` (the condition), `body` (the execution block if true), and `orelse` (the execution block if false).² Python's AST does not possess a distinct node for `elif` statements; instead, an `elif` is represented as an additional `ast.If` node nested directly within the `orelse` attribute of the preceding `ast.If`.²

To map this to DRAKON IR, the `ast.If` node translates to a decision primitive. The `test` attribute dictates the text field of the decision node. The `body` nodes generate subsequent action primitives that are linked via the `yes` pointer, continuing straight down the central Skewer.¹ The `orelse` elements are linked via the `no` pointer, directing the flow rightward. Because `elif` statements are nested in the `orelse` block, they recursively generate a chain of decision nodes expanding horizontally to the right. Finally, all divergent paths must converge at a Common Fate Merge point—a shared subsequent node that restores the unified downward flow.¹

Mapping Iteration Boundaries (For/While)

Iteration in Python is represented by `ast.For` and `ast.While` nodes, both of which contain `test/iter`, `body`, and `orelse` attributes.² DRAKON formalizes loops using paired macro-icons: `loop_start` (or `foreach`) and `loop_end`, establishing a clear boundary with a single entrance and potentially multiple exits.¹

The `ast.For` or `ast.While` declaration maps to the `loop_start` node. The statements within the `body` follow sequentially. The termination of the `body` generates a `loop_end` node, whose next pointer routes back to the `loop_start`, creating the iterative cycle. When the AST traversal encounters an `ast.Break` node, it generates an action block whose next pointer bypasses the `loop_end` entirely, routing forward to the first node following the loop.² Conversely, an

ast.Continue node generates a pointer that links directly back to the loop_start node.

Mapping Exception Handling (Try/Except/Finally)

The ast.Try node is complex, comprising body, handlers (a list of ast.ExceptHandler nodes), or_else, and finalbody attributes.² This construct is heavily governed by DRAKON's Rightward Degradation heuristic, which dictates that fatal errors and exception handlers must be forcibly pushed to the far right periphery of the diagram, visually isolating the risk zones.¹

The body of the try block executes along the main Skewer. However, because exceptions can occur implicitly, the extraction engine must inject a synthetic decision node immediately following the risky operation to represent the exception trap. The yes pointer of this synthetic decision (indicating an error was caught) routes rightward to the ExceptHandler blocks, which are mapped as action nodes. The no pointer continues down the Happy Path. The finalbody (the finally block) acts as the absolute convergence point; regardless of whether an exception occurred, all execution paths must route their next pointers to the finally block before the algorithm concludes.¹

Mapping Context Managers (With)

Python's with statements, represented by the ast.With node, serve as safe execution wrappers for resource management. In DRAKON semantics, the initialization of the context manager maps to a standard action node (e.g., Enter context: open(file)). The internal statements flow conventionally down the vertical axis. The exit from the context manager requires the insertion of an implicit action node (e.g., Exit context) to explicitly visualize the resource release before the algorithm proceeds.

Handling Asynchronous Routines and Generators

The introduction of asynchronous control flow (ast.AsyncFunctionDef, ast.Await) and generators (ast.Yield) modifies execution timing but does not fundamentally alter the spatial topology of the Control Flow Graph.

- **Asynchronous Await:** The await keyword is treated as an inline operation and embedded directly into the text of the corresponding action node. To provide visual differentiation, the platform may optionally map asynchronous calls to the specialized DRAKON process macro-icon, which denotes an asynchronous background task.¹
- **Yield Expressions:** While a return statement mandates a jump to the terminal End node of the algorithm, a yield statement merely suspends state. Therefore, ast.Yield maps to an action or output node whose next pointer simply connects to the subsequent sequential step, preserving the continuous flow of the generator.
- **Comprehensions:** List, set, and dictionary comprehensions (e.g., ast.ListComp) represent highly dense logic. Following the principle of AST simplification, comprehensions should not be exploded into complex loop_start/decision chains unless they contain deeply nested tertiary logic. Standard comprehensions map to a single

action node containing the comprehension's source text, minimizing visual clutter.

Execution Environments: Running Python Analysis in Cloudflare Workers

The current architecture of ai-drakon operates entirely within the Cloudflare Worker ecosystem, executing JavaScript within V8 isolates. The requirement to parse Python syntax and generate ASTs introduces a significant computational challenge. Four distinct architectural options present varying trade-offs regarding latency, accuracy, and infrastructure complexity.

Option A: Server-Side Python Microservice Backend

This architecture completely offloads the parsing workload from the edge. The Cloudflare Worker acts as a proxy, transmitting the raw Python source code via an HTTP POST request to a dedicated backend microservice written in Python (e.g., FastAPI). The microservice utilizes Python's native ast library or specialized packages like pyflowchart⁵ or code2flow¹⁰ to construct the CFG, outputting the formatted DRAGON IR JSON back to the Worker.

This approach offers the highest possible parsing fidelity. Python's native ast module is continually updated alongside the language, natively supporting structural pattern matching (match/case) introduced in Python 3.10 and newer syntax without requiring third-party library updates.² Furthermore, libraries like code2flow provide heuristic-based namespace resolution across multiple files, enabling deep cross-module call graph generation.¹ However, this option introduces severe architectural drawbacks. It breaks the pure serverless edge paradigm of the ai-drakon platform, introduces network latency for every file analyzed, and creates a centralized infrastructure bottleneck that requires continuous scaling and maintenance.

Option B: WebAssembly (WASM) via Tree-Sitter (Recommended Approach)

The second architecture involves compiling the tree-sitter core in C/Rust and the tree-sitter-python grammar into a WebAssembly (.wasm) binary. This binary is executed directly within the JavaScript Cloudflare Worker using the web-tree-sitter bindings.¹¹

Cloudflare Workers natively support importing .wasm files as WebAssembly.Module objects, which can be instantiated synchronously during the Worker's initialization phase.¹¹

JavaScript

```
import treeSitterPython from './tree-sitter-python.wasm';  
const Python = await Parser.Language.load(treeSitterPython);
```

Historically, Cloudflare Workers imposed a strict 1MB size limit on compiled assets, which restricted the deployment of large WASM modules.¹² However, modern Worker environments accommodate larger compressed bundles, allowing the ~1.5MB tree-sitter-python.wasm binary to be deployed successfully.

The advantages of this approach are paramount. Tree-Sitter provides an incredibly robust, error-tolerant parsing engine. Unlike Python's native ast module, which fails catastrophically upon encountering syntax errors, Tree-Sitter is designed for incremental parsing and handles broken code gracefully, producing a complete Concrete Syntax Tree (CST).¹ This is invaluable when analyzing incomplete or legacy codebases. By running entirely at the edge, this architecture eliminates external network calls, preserves the serverless topology, and ensures low-latency execution. The complexity lies primarily in the initial CI/CD setup to compile and bundle the WASM artifacts, but the operational benefits make this the definitively recommended approach.

Option C: Pure JavaScript Python Parsers

This architecture attempts to perform AST extraction using pure JavaScript libraries, such as filbert or skulpt, running natively within the Worker without WASM.

While this avoids the payload size concerns of WebAssembly and integrates seamlessly into the JS environment, it suffers from critical accuracy degradation. Pure JavaScript parsers for Python are notoriously difficult to maintain and frequently lag years behind the official Python language specifications. They struggle profoundly with modern syntactic sugar, type hinting annotations, walrus operators (`:=`), and asynchronous keywords. Using a pure JS parser would result in high failure rates when analyzing modern Python 3.10+ repositories, making this option fundamentally unviable for a production-grade refactoring platform.

Option D: Server-Side Pre-processing (CI/CD Integration)

The final option shifts the computational burden away from the ai-dragon runtime entirely. Abstract syntax tree extraction occurs locally on the user's machine or within a GitHub Actions CI/CD pipeline. The pipeline generates the AST JSON and pushes it to a storage bucket, which the Cloudflare Worker merely reads.

While this reduces operational costs for the platform, it destroys the dynamic nature of the tool. The dragon.analyzecodebase MCP tool expects to analyze arbitrary GitHub repositories on demand based on a user's prompt. Forcing users to configure complex webhook integrations and CI/CD pipelines before the AI agent can visualize their code introduces an unacceptable amount of friction.

Execution Environment Summary

The following table summarizes the comparative analysis of the execution environments for Python AST extraction within the ai-dragon platform:

Evaluation Criteria	Option A: Python Microservice	Option B: WASM Tree-Sitter	Option C: Pure JS Parser	Option D: CI/CD Pre-processing
Architectural Paradigm	Centralized Backend	Serverless Edge	Serverless Edge	Distributed Offline
Parsing Fidelity	Absolute (Native ast)	Extremely High (CST)	Low (Outdated Syntax)	Variable
Error Tolerance	Fails on Syntax Errors	Graceful Recovery	Fails on Syntax Errors	Fails on Syntax Errors
Latency	High (Network I/O)	Ultra-Low (Local V8)	Low (Local V8)	N/A (Asynchronous)
Recommendation	Secondary Fallback	Primary Recommendation	Not Recommended	Not Recommended

DRAKON Mapping Rules for Python-Specific Patterns

Translating raw Python code into the structural confines of the DRAKON Intermediate Representation requires adhering to strict invariant mapping rules. The system must algorithmically enforce the Single Terminal Constraint (all algorithms must end at exactly one terminator node) and the Vector Constraints for Bifurcations (conditional nodes exit exclusively downward and rightward).¹ The following analyses detail the precise logic for mapping the four requested Python patterns into flat JSON dictionaries.

Pattern 1: Sequential Conditional Chains (If/Elif/Else)

The evaluation of sequential conditional logic presents a topological challenge. The Python syntax implies a linear evaluation, but the AST structures elif as a nested entity within the orelse

branch.²

Python Source:

Python

```
if condition_a:
    action_a()
elif condition_b:
    action_b()
else:
    action_c()
```

Mapping Logic: The primary if maps to a decision node. The successful evaluation (condition_a) routes downward via the yes pointer to action_a. The failure route routes rightward via the no pointer to a second decision node (condition_b). This creates a cascading horizontal expansion. Crucially, action_a, action_b, and action_c must all define their next pointers to point to identical downstream identifier, satisfying the Common Fate Merge requirement.¹

DRAKON IR Transformation:

JSON

```
{
  "name": "Sequential Conditions",
  "items": {
    "1": { "type": "decision", "text": "condition_a?", "yes": "2", "no": "3" },
    "2": { "type": "action", "text": "action_a()", "next": "6" },
    "3": { "type": "decision", "text": "condition_b?", "yes": "4", "no": "5" },
    "4": { "type": "action", "text": "action_b()", "next": "6" },
    "5": { "type": "action", "text": "action_c()", "next": "6" },
    "6": { "type": "action", "text": "Post-conditional Logic", "next": "7" }
  }
}
```

Pattern 2: Iteration with Internal Exits (For Loop with Break)

Python loops combined with internal control flow manipulation (break) violate strict linear progression. In DRAKON, loops must be bounded by macroscopic icons to clarify scope.

Python Source:

Python

```
for item in collection:
    if predicate(item):
        break
    process(item)
```

Mapping Logic: The for statement initializes a loop_start node. The body begins with a decision node representing the predicate(item). The true branch of this decision executes the break command. In the IR, break does not point to the loop boundary; its next pointer acts as an absolute goto, targeting the node positioned chronologically after the loop_end. The false branch processes the item and points to loop_end, which automatically cycles back to loop_start.¹

DRAKON IR Transformation:

JSON

```
{
  "name": "Iteration with Break",
  "items": {
    "1": { "type": "loop_start", "text": "for item in collection", "next": "2" },
    "2": { "type": "decision", "text": "predicate(item)?", "yes": "3", "no": "4" },
    "3": { "type": "action", "text": "break", "next": "6" },
    "4": { "type": "action", "text": "process(item)", "next": "5" },
    "5": { "type": "loop_end", "text": "End loop", "next": "1" },
    "6": { "type": "action", "text": "Execution resumes here", "next": "7" }
  }
}
```

Pattern 3: Exception Trapping (Try/Except/Finally)

Exception handling is the primary use case for the Rightward Degradation heuristic. DRAKON

philosophy dictates that the engineer's eye must instantly recognize the far-right zone as the risk mitigation area.¹

Python Source:

Python

```
try:
    risky_operation()
except SpecificError as e:
    handle_error(e)
finally:
    cleanup()
```

Mapping Logic: Because `risky_operation()` might fail internally without a visible conditional operator, the mapping engine injects a synthetic decision immediately after the action. To enforce Rightward Degradation, the yes pointer (indicating the exception was caught) routes horizontally to the right into the `ExceptionHandler` block. The no pointer continues vertically down the main Skewer. Both the error handler and the main Skewer must converge their next pointers onto the `finally` block before proceeding.¹

DRAKON IR Transformation:

JSON

```
{
  "name": "Exception Handling",
  "items": {
    "1": { "type": "action", "text": "risky_operation()", "next": "2" },
    "2": { "type": "decision", "text": "SpecificError caught?", "yes": "4", "no": "3" },
    "3": { "type": "action", "text": "(Happy Path)", "next": "5" },
    "4": { "type": "action", "text": "handle_error(e)", "next": "5" },
    "5": { "type": "action", "text": "cleanup() [Finally]", "next": "6" }
  }
}
```

Pattern 4: Nested Class Methods and Early Returns

Complex methods often contain string manipulation, iterative searches, and multiple exit points. The DRAKON engine must normalize these structures into a compliant flat graph.

Python Source:

Python

```
class SlotRouter:
    def resolve_slot(self, model_name: str) -> ProxySlotConfig | None:
        normalized = model_name.strip()
        if "/" in normalized:
            suffix = normalized.rsplit("/", 1)[-1]
            normalized = suffix or normalized
        normalized = self._ALIASES.get(normalized, normalized)
        for slot in self._slots_loader():
            if slot.slot_id == normalized:
                if slot.mode == "disabled":
                    return None
                return slot
        return None
```

Mapping Logic: The class wrapper SlotRouter is omitted as a physical node but is prepended to the diagram's title metadata. The series of string assignments (normalized =..., suffix =...) are compressed into singular action nodes to minimize graphical noise.⁵

The most critical mapping rule applied here is the Single Terminal Constraint.¹ The Python code contains three distinct return statements. In DRAKON, an algorithm is forbidden from having multiple terminator endpoints.¹ The parser must intercept every return statement, map them to action nodes containing the return text, and enforce that all of their next pointers target a single, shared terminator node with the text "END".¹

DRAKON IR Transformation:

JSON

```
{
  "name": "SlotRouter.resolve_slot",
  "items": {
```

```

"1": { "type": "terminator", "text": "START", "next": "2" },
"2": { "type": "action", "text": "normalized = model_name.strip()", "next": "3" },
"3": { "type": "decision", "text": "'/' in normalized?", "yes": "4", "no": "5" },
"4": { "type": "action", "text": "suffix = normalized.rsplit...\n\nnormalized = suffix...", "next": "5" },
"5": { "type": "action", "text": "normalized = self._ALIASES.get...", "next": "6" },
"6": { "type": "loop_start", "text": "for slot in self._slots_loader()", "next": "7" },
"7": { "type": "decision", "text": "slot.slot_id == normalized?", "yes": "8", "no": "11" },
"8": { "type": "decision", "text": "slot.mode == 'disabled'?", "yes": "9", "no": "10" },
"9": { "type": "action", "text": "return None", "next": "13" },
"10": { "type": "action", "text": "return slot", "next": "13" },
"11": { "type": "loop_end", "text": "End Loop", "next": "12" },
"12": { "type": "action", "text": "return None", "next": "13" },
"13": { "type": "terminator", "text": "END", "next": null }
}
}

```

Complexity Estimation and Architectural Thresholds

Not every Python function merits visualization. Attempting to parse and render trivial getters and setters produces extreme visual noise, directly violating the core DRAKON principle:

"Clarity is above all".¹ The automated pipeline must employ rigorous mathematical models to evaluate AST complexity and decide whether a function should be visualized as a Primitive, decomposed into a Silhouette, or bypassed entirely.¹

Cyclomatic Complexity Assessment

McCabe's Cyclomatic Complexity, denoted as $V(G)$, provides a quantitative measure of the number of linearly independent paths through a program. For Python ASTs, this is calculated by evaluating the edge (E), node (N), and connected component (P) relationships:

$$V(G) = E - N + 2P$$

In practical AST parsing, this is approximated by establishing a base score of 1 and adding 1 for every branching node encountered (ast.If, ast.For, ast.While, ast.ExceptHandler, and boolean operators like and/or).

The platform enforces the following thresholds based on $V(G)$:

- $V(G) \leq 2$ **(Trivial)**: The function is too simple for standalone visualization. The system records its signature but represents it merely as an action or call node when referenced by parent functions.

- $3 \leq V(G) \leq 10$ **(Standard)**: The algorithm exhibits moderate complexity and should be generated as a standard DRAKON **Primitive** diagram (a single continuous Skewer).¹
- $V(G) > 10$ **(High Complexity)**: Rendering a Primitive would result in an illegible "cobweb" of intersecting logic. The parser must automatically trigger algorithm decomposition, slicing the logic into parallel chronological branches to form a **Silhouette** macro-template.¹

Evaluative Metrics: SIS and RDC

Beyond standard cyclomatic complexity, the platform utilizes two DRAKON-specific proprietary metrics to evaluate the structural integrity of the generated IR: the Skewer Integrity Score and the Rightward Degradation Count.¹

The **Skewer Integrity Score (SIS)** calculates the concentration of algorithmic logic residing on the central "Happy Path". It is defined mathematically as the ratio of nodes connected exclusively via the one (or yes pointing downward) traversal chain to the total number of nodes in the diagram:

$$SIS = \frac{\sum Nodes_{MainSkewer}}{\sum Nodes_{Total}}$$

An optimal DRAKON diagram exhibits an $SIS > 0.40$. If the SIS falls below 0.20, the algorithm is excessively fragmented, indicating poor code refactoring or an urgent need for Silhouette decomposition.

The **Rightward Degradation Count (RDC)** measures the maximum spatial nesting depth along the horizontal X-axis. It is computed by tracking the longest continuous chain of transitions utilizing the two (or no) pointer.¹ If $RDC \geq 4$, the cognitive load on the reader exceeds acceptable ergonomic limits. The parser must isolate these deep conditional chains and extract them as external subroutines using the insertion (call) macro-icon.¹

Incremental Analysis Strategy for Large Codebases

When the `drakon.analyzecodebase` MCP tool is invoked on a modern Python repository containing 50 to 200 files, processing every AST node synchronously is computationally prohibitive. It will result in memory saturation and trigger execution timeout limits within the Cloudflare Worker infrastructure. To maintain performance, an incremental, priority-driven analysis strategy is required.

Prioritization and Dependency Resolution

The analyzer must construct a global module dependency graph before initiating intra-function

AST traversal. Files and functions are queued based on their topological importance:

1. **Entry Points:** Functions decorated with framework routing (e.g., FastAPI `@app.get`), files executing within `if __name__ == "__main__":` blocks, and core CLI initialization scripts receive maximum priority.
2. **High-Traffic Nodes:** Utility functions demonstrating high incoming edge counts within the global Call Graph are processed next, as their visualization is necessary to understand downstream logic.¹³
3. **Deferred Processing:** Abstract base classes, interface definitions, and pure data schemas (e.g., empty dataclasses or Pydantic models) are deferred or entirely excluded from flow visualization, as they lack control flow logic.

Cross-Module Reference Handling

When the AST parser encounters a function call (e.g., `ast.Call` invoking `auth.verify_token()`), it consults the global registry. If `verify_token` possesses a complexity score warranting a diagram, the parser maps the call to an insertion node.¹ The insertion node acts as a "black box" macro-icon. In the ai-drakon UI, double-clicking this node triggers a hyperlink event, dynamically loading the specific DRAGON schema for `verify_token`, thereby ensuring seamless hierarchical navigation through the codebase.¹

Edge-Optimized Caching Strategy

Because Cloudflare Workers execute transiently, analyzing 200 files repeatedly is unsustainable. The system implements a dual-layer caching strategy utilizing Cloudflare KV and MinIO object storage.

- **Source Hashing:** Upon fetching a file via the GitHub API, the Worker computes an SHA-256 hash of its text. It checks Cloudflare KV for the key `ast:sha256:<hash>`. If a match exists, the file has not mutated, and the costly WASM AST extraction phase is bypassed entirely.
- **IR Storage:** The resulting flat JSON dictionaries are persisted to MinIO under paths corresponding to their repository slug and function IDs (e.g., `drakon/${repo_slug}/${diagram_id}.json`).¹ During cross-module reference resolution, the Worker fetches these pre-computed JSON files directly from storage rather than re-parsing the target module.

Target System Architecture Pipeline

The integration of the WebAssembly Tree-Sitter parser within the Cloudflare Worker completes the HTSE pipeline, enabling fully autonomous, serverless code-to-diagram generation.

Pipeline Execution Flow

1. **Invocation:** The `drakon.analyzecodebase` MCP tool is called by an AI agent or UI trigger, passing the target owner/repo parameters.

2. **Fetch & Cache Check:** The Worker retrieves the repository files via the GitHub API. It calculates SHA-256 hashes and cross-references them against Cloudflare KV to isolate modified or new Python files.
3. **WASM Instantiation:** For files requiring analysis, the Worker synchronously instantiates the tree-sitter-python.wasm module using the web-tree-sitter bindings within the V8 isolate.¹¹
4. **CST Extraction:** The WASM engine parses the Python source text, generating a robust Concrete Syntax Tree (CST) that gracefully tolerates minor syntax errors.¹¹
5. **CFG to IR Translation:** The JavaScript-based DrakonTranslator (detailed in the reference implementation below) traverses the CST. It applies the Rightward Degradation heuristics, collapses trivial assignments, and calculates complexity metrics ($V(G)$, SIS, RDC).
6. **Validation:** The generated flat JSON dictionary is passed through the internal Graph Validator microservice. This deterministic module enforces the Single Terminal Constraint and prunes any "orphan" nodes.¹
7. **Persistence & Rendering:** The validated JSON IR is pushed to MinIO object storage.¹ The ai-drakon frontend fetches the JSON, passing it to drakonwidget.js, which dynamically executes Manhattan routing and renders the visual flowcharts in the browser.¹

Complete Mapping Table: Python to DRAKON Node

The following table provides the definitive semantic mapping constraints utilized by the DrakonTranslator engine during the CST traversal phase:

Python Syntax Construct (AST Node)	DRAKON IR Node Type	Topological / Geometric Mapping Rule
def func() / async def / __main__	terminator (START)	Anchored at the absolute top of the primary vertical Skewer.
return / sys.exit() / raise	terminator (END)	Converged to a singular, shared END node at the bottom right. ¹
a = b, print(), await, pass	action	Placed sequentially along the downward vertical axis.
if cond: / elif: / match	decision	yes routes downward; no routes strictly horizontally

		to the right.
for i in X:, while cond:	loop_start	Placed on the vertical axis; initializes the iterative boundary.
End of loop body	loop_end	Closes the loop; generates a reverse pointer back to loop_start.
break	action	Bypasses loop_end, jumping directly to the post-loop sequential node.
try:	action (Wrapper)	Indicates entry into a risk zone; flow continues down the Skewer.
except Exception as e:	decision (Synthetic)	Evaluated immediately after risk; routes horizontally into error branch.
finally:	action (Convergence)	Acts as the Common Fate Merge point for success and error paths.
with context:	action	Context enter/exit are collapsed into a single functional block.
func_call() (External Module)	call (Insertion)	Renders as a macro-icon allowing interactive drill-down. ¹

Reference Implementation: CST to IR Translation

The following conceptual JavaScript implementation demonstrates the logic of traversing a Tree-Sitter Python CST and constructing the mathematically precise, flat JSON adjacency list required by drakonwidget.js.

JavaScript

```
/**
 * DrakonTranslator: Converts Tree-Sitter Python CST to DRAKON Flat JSON IR.
 * Designed for execution within Cloudflare Workers edge environment.
 */
class DrakonTranslator {
  constructor() {
    this.items = {};
    this.nodeCounter = 1;
  }

  generateId() {
    return (this.nodeCounter++).toString();
  }

  // Orchestrator for function/method definitions
  translateFunctionDef(functionNode) {
    const startId = this.generateId();
    const endId = "9999"; // Reserved to enforce Single Terminal Convergence

    this.items[startId] = {
      type: "terminator",
      text: `START: ${functionNode.childForFieldName('name').text}`,
      next: null
    };

    const bodyNode = functionNode.childForFieldName('body');
    const lastId = this.traverseBlock(bodyNode, startId, endId);

    // Enforce convergence: Link the final node of the Skewer to the terminal node
    if (this.items[lastId] && !this.items[lastId].next) {
      this.items[lastId].next = endId;
    }

    this.items[endId] = {
      type: "terminator",
      text: "END",
      next: null
    };

    return {
```

```

    name: functionNode.childForFieldName('name').text,
    items: this.items
  };
}

// Sequentially traverses a block, applying AST simplification
traverseBlock(blockNode, parentId, endId) {
  let currentParentId = parentId;
  let assignmentBuffer =; // Buffer for collapsing sequential assignments

  for (let i = 0; i < blockNode.childCount; i++) {
    const stmt = blockNode.child(i);

    // Simplify: Collapse consecutive non-branching statements into one action
    if (stmt.type === 'expression_statement' |

| stmt.type === 'assignment') {
      assignmentBuffer.push(stmt.text.substring(0, 50));
      continue;
    }

    // Flush the buffer before evaluating complex control flow
    if (assignmentBuffer.length > 0) {
      currentParentId = this.flushBuffer(assignmentBuffer, currentParentId);
      assignmentBuffer =;
    }

    if (stmt.type === 'if_statement') {
      currentParentId = this.translateIfStatement(stmt, currentParentId, endId);
    }
    else if (stmt.type === 'return_statement') {
      const actionId = this.generateId();
      this.items[actionId] = {
        type: "action",
        text: `return ${stmt.childForFieldName('return_value')?.text |

| "`,
        next: endId // Routes directly to the global end node
      };
      this.items[currentParentId].next = actionId;
      currentParentId = actionId;
    }
  }

  // Handlers for for_statement, while_statement, try_statement operate similarly

```

```
}
```

```
// Final buffer flush at block termination
```

```
if (assignmentBuffer.length > 0) {  
    currentParentId = this.flushBuffer(assignmentBuffer, currentParentId);  
}
```

```
return currentParentId;  
}
```

```
flushBuffer(buffer, parentId) {  
    const actionId = this.generateId();  
    this.items[actionId] = {  
        type: "action",  
        text: buffer.join('\n'), // Multi-line text block  
        next: null  
    };  
    this.items[parentId].next = actionId;  
    return actionId;  
}
```

```
// Handles Rightward Degradation logic for conditional bifurcations
```

```
translateIfStatement(ifNode, parentId, globalEndId) {  
    const decisionId = this.generateId();  
    const conditionText = ifNode.childForFieldName('condition').text;
```

```
    this.items[decisionId] = {  
        type: "decision",  
        text: `${conditionText}?`,  
        yes: null,  
        no: null  
    };  
    this.items[parentId].next = decisionId;
```

```
// Happy Path: Routes downward on the primary vertical Skewer
```

```
const consequenceNode = ifNode.childForFieldName('consequence');  
const consequenceLastId = this.traverseBlock(consequenceNode, decisionId, globalEndId);  
this.items[decisionId].yes = this.items[decisionId].next;  
delete this.items[decisionId].next; // 'decision' relies solely on yes/no pointers
```

```
// Alternative Path: Routes horizontally to the right
```

```
const alternativeNode = ifNode.childForFieldName('alternative');  
let alternativeLastId = null;
```

```

if (alternativeNode) {
  alternativeLastId = this.traverseBlock(alternativeNode, decisionId, globalEndId);
  this.items[decisionId].no = this.items[decisionId].next;
} else {
  // Missing 'else' mandates a pass node routing back to the Skewer
  const passId = this.generateId();
  this.items[passId] = { type: "action", text: "pass", next: null };
  this.items[decisionId].no = passId;
  alternativeLastId = passId;
}

// Common Fate Merge: Both branches must converge to continue
const mergeld = this.generateId();
this.items[mergeld] = { type: "action", text: "Convergence", next: null };

this.items[consequenceLastId].next = mergeld;
if (alternativeLastId) this.items[alternativeLastId].next = mergeld;

return mergeld;
}
}

```

The adoption of this architecture—combining edge-native WebAssembly AST extraction with a deterministic translation engine—completely resolves the spatial hallucinations inherent to pure LLM generation. It guarantees that the resulting JSON IR perfectly mirrors the Python source code while strictly adhering to the cognitive ergonomics and topological invariants of the DRAKON visual language.

Джерела

1. Специфікація інженерної системи DRAKON.pdf
2. ast — Abstract syntax trees — Python 3.14.5 documentation, доступ отримано травня 12, 2026, <https://docs.python.org/3/library/ast.html>
3. Analyzing Python Codes with AST (1) | by Ken Maeda | Medium, доступ отримано травня 12, 2026, <https://medium.com/@whiteking64/analyzing-python-codes-with-ast-1-cd15e97dd79a>
4. Python ast transformation - klara 0.6.3 documentation, доступ отримано травня 12, 2026, https://klara-py.readthedocs.io/en/latest/cfg_ssa.html
5. cdfmlr/pyflowchart: Python codes to Flowcharts - GitHub, доступ отримано травня 11, 2026, <https://github.com/cdfmlr/pyflowchart>
6. pyflowchart streamlit + cheat sheet - GitHub Gist, доступ отримано травня 12, 2026, <https://gist.github.com/kaigouthro/fd9aed52895a5668f6059f5e9d93f5a8>

7. What is ast.If(test, body, orelse) in Python? - Educative.io, доступ отримано травня 12, 2026,
<https://www.educative.io/answers/what-is-astiftest-body-orelse-in-python>
8. DRAKON Flowchart Tutorial Part 1 | PDF | Algorithms And Data Structures - Scribd, доступ отримано травня 11, 2026,
<https://www.scribd.com/document/465913032/drakon-part1-eng-pdf>
9. Python try...except...finally Statement - Python Tutorial, доступ отримано травня 11, 2026, <https://www.pythontutorial.net/python-basics/python-try-except-finally/>
10. py2cfg - PyPI, доступ отримано травня 12, 2026, <https://pypi.org/project/py2cfg/>
11. tree-sitter/lib/binding_web/README.md at master - GitHub, доступ отримано травня 11, 2026,
https://github.com/tree-sitter/tree-sitter/blob/master/lib/binding_web/README.md
12. Cloudflare workers · duckdb duckdb-wasm · Discussion #430 - GitHub, доступ отримано травня 12, 2026,
<https://github.com/duckdb/duckdb-wasm/discussions/430>
13. The Python Control Flow Graph - Rahul Gopinath, доступ отримано травня 12, 2026, <https://rahul.gopinath.org/post/2019/12/08/python-controlflow/>